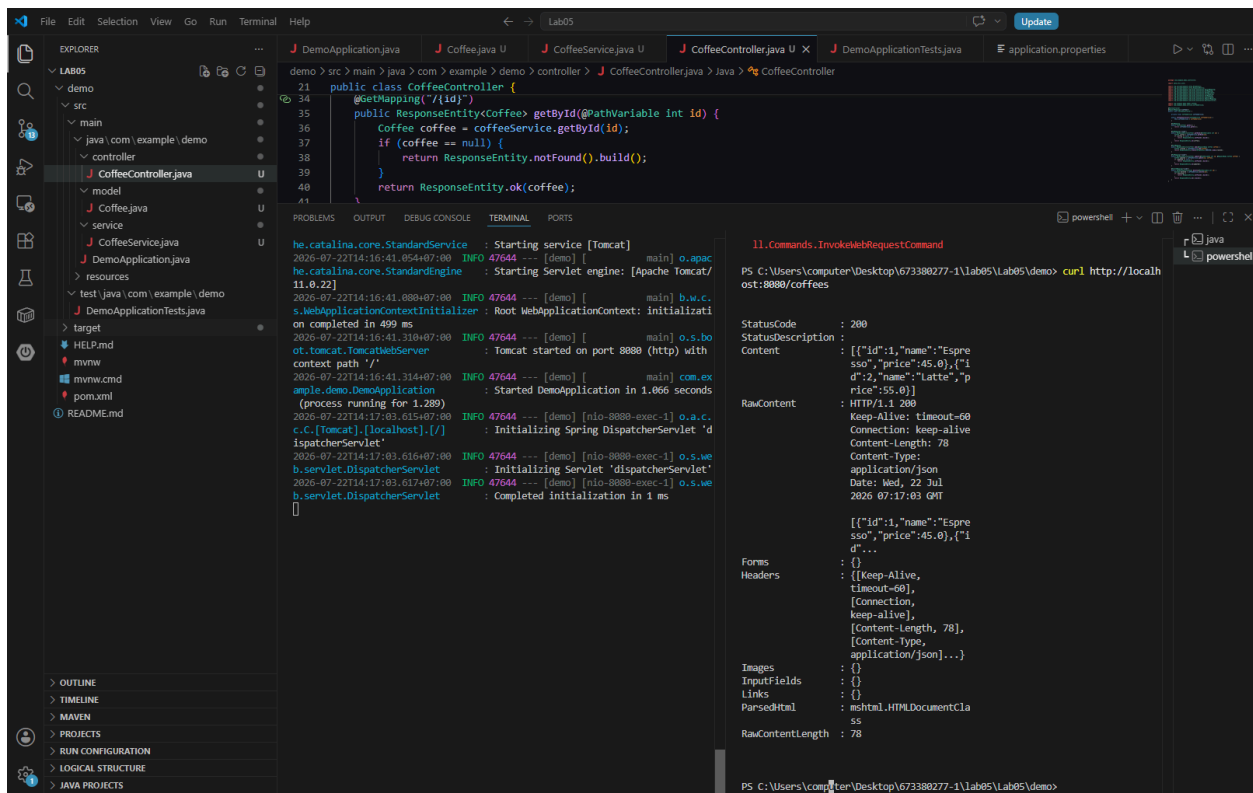




```
PS C:\Users\computer\Desktop\673380277-1\lab05\Lab05\demo> curl http://localh
ost:8080/caffe
```

```
StatusCode : 200
StatusDescription :
Content : [{"id":1,"name":"Espre
sso","price":45.0},{"i
d":2,"name":"Latte","p
rice":55.0}]
RawContent : HTTP/1.1 200
Keep-Alive: timeout=60
Connection: keep-alive
Content-Length: 78
Content-Type:
application/json
Date: Wed, 22 Jul
2026 07:17:03 GMT
```

```
[{"id":1,"name":"Espre
sso","price":45.0},{"i
d":...
Forms : {}
Headers : {[Keep-Alive,
timeout=60],
[Connection,
keep-alive],
[Content-Length, 78],
[Content-Type,
application/json]...}
Images : {}
InputFields : {}
Links : {}
ParsedHtml : mshtml.HTMLDocumentCla
ss
RawContentLength : 78
```

IDE screenshot showing the development of a Spring Boot application. The Explorer pane on the left shows the project structure: LAB05 > demo > src > main > java > com > example > demo > controller > CoffeeController.java. The main editor displays the code for CoffeeController.java, which includes a GET endpoint for retrieving coffee by ID and a POST endpoint for adding a new coffee. The terminal pane at the bottom shows the application running on port 8080, with logs indicating the start of the service and the initialization of the Spring DispatcherServlet. The output pane on the right shows the result of a curl command: a 200 status code and a JSON response containing coffee details.

```
demo > src > main > java > com > example > demo > controller > CoffeeController.java > Java > CoffeeController > update(int id, Coffee coffee)

21 public class CoffeeController {
22     @GetMapping("/{id}")
23     public ResponseEntity<Coffee> getById(@PathVariable int id) {
24         Coffee coffee = coffeeService.getById(id);
25         if (coffee == null) {
26             return ResponseEntity.notFound().build();
27         }
28         return ResponseEntity.ok(coffee);
29     }
30
31     @PostMapping
32     public ResponseEntity<Coffee> add(@RequestBody Coffee coffee) {
33         Coffee created = coffeeService.add(coffee);
34         return ResponseEntity.status(HttpStatus.CREATED).body(created);
35     }
36
37     @PutMapping("/{id}")
38     public ResponseEntity<Coffee> update(@PathVariable int id, @RequestBody Coffee coffee) {
39         Coffee updated = coffeeService.update(id, coffee);
40         if (updated == null) {
41             return ResponseEntity.notFound().build();
42         }
43         return ResponseEntity.ok(updated);
44     }
45 }
```

he.catalina.core.StandardService : Starting service [Tomcat]
2026-07-22T14:16:41.054+07:00 INFO 47644 --- [demo] [main] o.apac
he.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/
11.0.22]
2026-07-22T14:16:41.888+07:00 INFO 47644 --- [demo] [main] b.w.c.
s.WebApplicationContext: Initializing
on completed in 499 ms
2026-07-22T14:16:41.318+07:00 INFO 47644 --- [demo] [main] o.s.bo
ot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with
context path '/'
2026-07-22T14:16:41.314+07:00 INFO 47644 --- [demo] [main] com.ex
ample.demo.DemoApplication : Started DemoApplication in 1.066 seconds
(process running for 1.289)
2026-07-22T14:17:03.615+07:00 INFO 47644 --- [demo] [nio-8080-exec-1] o.a.c.
c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'd
ispatcherServlet'
2026-07-22T14:17:03.616+07:00 INFO 47644 --- [demo] [nio-8080-exec-1] o.s.we
b.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-22T14:17:03.617+07:00 INFO 47644 --- [demo] [nio-8080-exec-1] o.s.we
b.servlet.DispatcherServlet : Completed initialization in 1 ms

PS C:\Users\computer\Desktop\673380277-1\Lab05\Lab05\demo> curl http://localh
ost:8080/coffees/1

StatusCode : 200
StatusDescription :
Content : {"id":1,"name":"Espresso","price":45.0}
RawContent : HTTP/1.1 200
Transfer-Encoding: chunked
Keep-Alive: timeout=60
Connection: keep-alive
Content-Type: application/json
Date: Wed, 22 Jul 2026 07:20:36 GMT

Forms : {}
Headers : {[Transfer-Encoding, chunked], [Keep-Alive,
timeout=60], [Connection, keep-alive], [Content-Type,
application/json]...}
Images : {}
InputFields : {}
Links : {}
ParsedHtml : mshtml.HTMLDocumentClass
RawContentLength : 39

PS C:\Users\computer\Desktop\673380277-1\Lab05\Lab05\demo>

```
PS C:\Users\computer\Desktop\673380277-1\lab05\Lab05\demo> curl http://localhost:8080/coffees/1
```

```
StatusCode      : 200
StatusDescription :
Content         : {"id":1,"name":"Espresso","price":45.0}
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Keep-Alive: timeout=60
                  Connection: keep-alive
                  Content-Type: application/json
                  Date: Wed, 22 Jul 2026 07:20:36 GMT
```

```
{"id":1,"name":"Espresso","price":45.0...
Forms           : {}
Headers         : {[Transfer-Encoding, chunked], [Keep-Alive,
                  timeout=60], [Connection, keep-alive], [Content-Type,
                  application/json]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 39
```

HTTP  http://localhost:8080/coffees  Save

POST  http://localhost:8080/coffees  Send 

Params Authorization Headers (8) **Body**  Pre-request Script Tests Settings Cookies

 none  form-data  x-www-form-urlencoded  raw  binary JSON  Beautify

```
1 { "name": "Cappuccino", "price": 60.0 }
```

Body Cookies Headers (5) Test Results  201 Created 126 ms 210 B  Save Response 

Pretty Raw Preview Visualize JSON  

```
1 {
2   "id": 3,
3   "name": "Cappuccino",
4   "price": 60.0
5 }
```

POST http://localhost:8080/coffees PUT http://localhost:8080/coffees/2 + ...

HTTP  http://localhost:8080/coffees/2  Save

PUT  http://localhost:8080/coffees/2  Send 

Params Authorization Headers (8) **Body**  Pre-request Script Tests Settings Cookies

 none  form-data  x-www-form-urlencoded  raw  binary JSON  Beautify

```
1 { "name": "Latte", "price": 50.0 }
```

Body Cookies Headers (5) Test Results  200 OK 8 ms 200 B  Save Response 

Pretty Raw Preview Visualize JSON  

```
1 {
2   "id": 2,
3   "name": "Latte",
4   "price": 50.0
5 }
```

POST http://localhost:8080/co PUT http://localhost:8080/cofi DEL http://localhost:8080/cofi + ...



http://localhost:8080/coffees/3

Save

DELETE



http://localhost:8080/coffees/3

Send



Params Authorization Headers (6) Body Pre-request Script Tests Settings

Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (4) Test Results

200 OK 5 ms 123 B Save Response

Pretty

Raw

Preview

Visualize

Text



1

POST http://localhost:8080/co PUT http://localhost:8080/cofi DEL http://localhost:8080/cofi GET http://localhost:8080/cofi + ...



http://localhost:8080/coffees

Save

GET



http://localhost:8080/coffees

Send



Params Authorization Headers (6) Body Pre-request Script Tests Settings

Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (5) Test Results

200 OK 7 ms 234 B Save Response

Pretty

Raw

Preview

Visualize

JSON



```
3     "id": 1,  
4     "name": "Espresso",  
5     "price": 45.0  
6   },  
7   {  
8     "id": 2,  
9     "name": "Latte",  
10    "price": 50.0  
11  }  
12 ]
```

1. HTTP method แต่ละตัวต่างกันอย่างไร

ต่างกันที่หน้าที่การจัดการข้อมูลครับ สรุปง่าย ๆ คือ GET ใช้ดึงข้อมูลมาดู, POST ใช้ส่งข้อมูลเข้าไปเพิ่มใหม่, PUT ใช้แก้ไขหรืออัปเดตข้อมูลเดิม และ DELETE ใช้ลบข้อมูลทิ้ง (เช่น GET ดูเมนูกาแฟ, POST เพิ่มเมนูใหม่)

GET = ดู, POST = เพิ่ม, PUT = แก้ไข, DELETE = ลบ

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน

เพื่อความเป็นระเบียบและลดความซับซ้อน ให้ Controller ทำหน้าที่แค่รับ Request แล้วส่งต่อให้ Service จัดการลอจิกเบื้องหลัง ถ้าเอาไปยัดรวมกันหมดพอโปรเจกต์ใหญ่ขึ้นโค้ดจะรก การแยกกันทำให้เราแก้โค้ดและเขียนเทสทำได้ง่ายกว่าเยอะครับ

3. ข้อมูลใน List หายตอนไหน และถ้าไม่อยากให้หายต้องทำยังไง

ข้อมูลจะหายไปตอนที่เรารันโปรแกรมหรือรีสตาร์ทเซิร์ฟเวอร์ เพราะมันถูกเก็บไว้ชั่วคราวใน RAM เท่านั้น ถ้าไม่อยากให้ข้อมูลหาย ต้องเปลี่ยนไปบันทึกลงที่เก็บข้อมูลถาวร เช่น เซฟเป็นไฟล์ หรือบันทึกลงฐานข้อมูลอย่าง MySQL

4. Annotation แต่ละตัวทำหน้าที่อะไร

`@RestController` : บอกว่าคลาสนี้เป็น API (ตอบกลับเป็น JSON)

`@GetMapping` / `@PostMapping` : กำหนด URL ว่าถ้ามีคนเรียก GET หรือ POST เข้ามา ให้มาทำงานที่ฟังก์ชันนี้

`@PathVariable` : ไว้ดึงค่าตัวแปรที่แนบมากับ URL (เช่น ดึง id ออกมา)

`@RequestBody` : ใช้แปลงข้อมูล JSON ที่ผู้ใช้ส่งมาใน Body ให้กลายเป็น Object ของ Java